

Documentation de l'API

Cette documentation de l'ensemble des endpoints de l'API est aussi disponible dans la partie 6 du readme : Voir ici

1 Authentication

Les endpoints d'authentification permettent la création de compte, la connexion et la récupération des informations de l'utilisateur connecté.

L'authentification repose sur des JSON Web Tokens (JWT) transmis via l'en-tête **Authorization**.

POST /auth/register

Description

Crée un nouveau compte utilisateur.

Authentication requise

Aucune.

Middleware

Aucun.

Validation (Zod)

- **email** : string valide au format email
- **firstname** : string de taille entre 3 et 30 caractères
- **lastname** : string de taille entre 3 et 30 caractères
- **password** : string de taille entre 6 et 255 caractères

Headers

Aucun.

Paramètres

Aucun.

Body attendu

```
{
  "email" : "test3@test.com",
  "firstname" : "Édouard",
  "lastname" : "Paul",
  "password" : "cypcyp"
}
```

Réponse – Succès (201)

[illegible]

Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod)
- 500 Internal Server Error : erreur interne du serveur

POST /auth/login

Description

Authentifie un utilisateur existant et retourne un token JWT permettant d'accéder aux routes protégées de l'API.

Authentication requisite

Aucune.

Middleware

Aucun.

Validation (Zod)

- email : string valide au format email
- password : string non vide de taille entre 6 et 255 caractères

Headers

Aucun.

Paramètres

Aucun.

Body attendu

```
{
  "email" : "test@test.com",
  "password" : "motdepasse"
}
```

Réponse – Succès (200)

```
{
  "message": "User logged in",
  "userData": {
    "id": "3e5ab941-2b2a-4f57-958d-a361a82628c2",
    "email": "test@test.com"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiIzZTVhYjkiOms0yYjJhLTRmNTctOTU4ZC1hMzYxYTgyNn0.e3BkbnQ"
}
```

Erreurs possibles

- 400 **Bad Request** : données invalides (échec de la validation Zod)
- 401 **Unauthorized** : email ou mot de passe incorrect
- 500 **Internal Server Error** : erreur interne du serveur

GET /auth/information

Description

Récupère les informations du compte de l'utilisateur actuellement authentifié.

Authentication requise

Oui — token JWT valide.

Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT

Validation (Zod)

Aucun.

Headers

- ```
- Authorization: Bearer <token>
```

## Paramètres

Aucun.

### Body attendu

Aucun.

### Réponse – Succès (200)

```
{
 "message": "User information :",
 "userData": {
 "id": "fca23035-97e9-4007-a296-9e8532183906",
 "firstname": "Édouard",
 "lastname": "Paul",
 "email": "test3@test.com"
 }
}
```

### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide
  - 404 Not Found : utilisateur non trouvé
  - 500 Internal Server Error : erreur interne du serveur
- 

## 2 Collections

Les endpoints Collections permettent de créer, consulter, modifier, supprimer et rechercher des collections de flash-cards.

Toutes les routes nécessitent **un utilisateur authentifié** via JWT.

---

### POST /collections/createCollection

#### Description

Crée une nouvelle collection de flashcards.

#### Authentication requise

Oui — JWT valide.

#### Middleware

- authenticateToken : vérifie la présence et la validité du JWT.
- validateBody : valide le corps de la requête.

#### Validation (Zod)

- title : string non vide de taille max 100.
- description : string non vide de taille max 512.
- is\_private : boolean (optionnel).

#### Headers

- Authorization: Bearer <token>

#### Paramètres

Aucun.

### Body attendu

```
{
 "title": "Ma nouvelle collection",
 "description": "Description de ma collection.",
 "is_private": false
}
```

### Réponse – Succès (201)

```
{
 "message": "Question created",
 "data": {
 "id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "title": "Ma nouvelle collection",
 "description": "Description de ma collection.",
 "user_id": "fca23035-97e9-4007-a296-9e8532183906",
 "is_private": false,
 "created_at": "2024-01-09T10:00:00.000Z",
 "updated_at": "2024-01-09T10:00:00.000Z"
 }
}
```

### Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod).
- 401 Unauthorized : token manquant, expiré ou invalide.
- 500 Internal Server Error : erreur interne du serveur.

---

## PATCH /collections/updateCollection/ :id

### Description

Modifie une collection existante (titre, description, visibilité).

### Authentification requise

Oui — JWT valide, et propriétaire de la collection ou admin.

### Middleware

- authenticateToken : vérifie la présence et la validité du JWT.
- validateParams : valide les paramètres de la route.
- validateBody : valide le corps de la requête.

### Validation (Zod)

- id : UUID (paramètre de route).
- title : string non vide de taille max 100 (optionnel).
- description : string non vide de taille max 512 (optionnel).
- is\_private : boolean (optionnel).

### Headers

- Authorization: Bearer <token>

### Paramètres

- id (route param, UUID) : identifiant de la collection.

### Body attendu

```
{
 "title": "Titre mis à jour"
}
```

### Réponse — Succès (200)

```
{
 "message": "Collection mise à jour",
 "data": {
 "id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "title": "Titre mis à jour",
 "description": "Description de ma collection.",
 "user_id": "fca23035-97e9-4007-a296-9e8532183906",
 "is_private": false,
 "created_at": "2024-01-09T10:00:00.000Z",
 }
}
```

```
 "updated_at": "2024-01-09T10:05:00.000Z"
 }
}
```

#### Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod).
  - 401 Unauthorized : token manquant, expiré ou invalide.
  - 403 Forbidden : l'utilisateur n'est pas le propriétaire ou un admin.
  - 404 Not Found : collection non trouvée.
  - 500 Internal Server Error : erreur interne du serveur.
- 

### GET /collections/collectionById/ :id

#### Description

Récupère une collection par son identifiant.

#### Authentification requise

Oui — JWT valide. Accès autorisé si la collection est publique, ou si l'utilisateur est propriétaire ou admin.

#### Middleware

- authenticateToken : vérifie la présence et la validité du JWT.
- validateParams : valide les paramètres de la route.

#### Validation (Zod)

- id : UUID (paramètre de route).

#### Headers

- Authorization: Bearer <token>

#### Paramètres

- id (route param, UUID) : identifiant de la collection.

#### Body attendu

Aucun.

#### Réponse – Succès (200)

```
{
 "id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "title": "Ma collection",
 "description": "Description de ma collection.",
 "user_id": "fca23035-97e9-4007-a296-9e8532183906",
 "is_private": false,
 "created_at": "2024-01-09T10:00:00.000Z",
 "updated_at": "2024-01-09T10:00:00.000Z"
}
```

#### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide.
  - 403 Forbidden : la collection est privée et l'utilisateur n'est pas autorisé.
  - 404 Not Found : collection non trouvée.
  - 500 Internal Server Error : erreur interne du serveur.
- 

### GET /collections/collectionFlashcards/ :id

#### Description

Récupère les flashcards d'une collection.

#### Authentification requise

Oui — JWT valide. Accès autorisé si la collection est publique, ou si l'utilisateur est propriétaire ou admin.

### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT.
- `validateParams` : valide les paramètres de la route.

### Validation (Zod)

- `id` : UUID (paramètre de route).

### Headers

- `Authorization`: Bearer <token>

### Paramètres

- `id` (route param, UUID) : identifiant de la collection.

### Body attendu

Aucun.

### Réponse – Succès (200)

```
[
 {
 "id": "f1c1b1a1-9f9d-4c8c-9c1c-8d8d1f1c9a0a",
 "front_text": "Recto de la carte",
 "back_text": "Verso de la carte",
 "collection_id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a"
 }
]
```

### Erreurs possibles

- 401 `Unauthorized` : token manquant, expiré ou invalide.
  - 403 `Forbidden` : la collection est privée et l'utilisateur n'est pas autorisé.
  - 404 `Not Found` : collection non trouvée ou pas de flashcards dans la collection.
  - 500 `Internal Server Error` : erreur interne du serveur.
- 

## GET /collections/collectionByTitle/ :title

### Description

Recherche des collections publiques par titre. Les collections privées de l'utilisateur sont aussi retournées.

### Authentification requise

Oui — JWT valide.

### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT.
- `validateParams` : valide les paramètres de la route.

### Validation (Zod)

- `title` : string non vide de taille max 100 (paramètre de route).

### Headers

- `Authorization`: Bearer <token>

### Paramètres

- `title` (route param, string) : expression à rechercher dans le titre.

### Body attendu

Aucun.

### Réponse – Succès (200)

```
[
 {
 "id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "title": "Ma collection",
 }
]
```

```

 "description": "Description de ma collection.",
 "user_id": "fca23035-97e9-4007-a296-9e8532183906",
 "is_private": false,
 "created_at": "2024-01-09T10:00:00.000Z",
 "updated_at": "2024-01-09T10:00:00.000Z"
 }
]

```

#### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide.
- 404 Not Found : aucune collection trouvée.
- 500 Internal Server Error : erreur interne du serveur.

### GET /collections/myCollection

#### Description

Liste toutes les collections appartenant à l'utilisateur connecté, avec leurs flashcards.

#### Authentification requise

Oui — JWT valide.

#### Middleware

- authenticateToken : vérifie la présence et la validité du JWT.

#### Validation (Zod)

Aucune.

#### Headers

- Authorization: Bearer <token>

#### Paramètres

Aucun.

#### Body attendu

Aucun.

#### Réponse – Succès (200)

```

[
 {
 "id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "title": "Ma collection",
 "description": "Description de ma collection.",
 "user_id": "fca23035-97e9-4007-a296-9e8532183906",
 "is_private": false,
 "created_at": "2024-01-09T10:00:00.000Z",
 "updated_at": "2024-01-09T10:00:00.000Z",
 "flashcards": [
 {
 "id": "f1c1b1a1-9f9d-4c8c-9c1c-8d8d1f1c9a0a",
 "front_text": "Recto de la carte",
 "back_text": "Verso de la carte",
 "collection_id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a"
 }
]
 }
]

```

#### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide.

- 404 Not Found : l'utilisateur n'a aucune collection.
  - 500 Internal Server Error : erreur interne du serveur.
- 

## **DELETE /collections/deleteCollection/ :id**

### **Description**

Supprime une collection existante et ses flashcards associées.

### **Authentication requisite**

Oui — JWT valide, propriétaire de la collection ou admin.

### **Middleware**

- `authenticateToken` : vérifie la présence et la validité du JWT.
- `validateParams` : valide les paramètres de la route.

### **Validation (Zod)**

- `id` : UUID (paramètre de route).

### **Headers**

- `Authorization`: Bearer <token>

### **Paramètres**

- `id` (route param, UUID) : identifiant de la collection.

### **Body attendu**

Aucun.

### **Réponse – Succès (200)**

```
{
 "message": "Collection deleted !"
}
```

### **Erreurs possibles**

- 401 Unauthorized : token manquant, expiré ou invalide.
  - 403 Forbidden : l'utilisateur n'est pas autorisé.
  - 404 Not Found : collection non trouvée.
  - 500 Internal Server Error : erreur interne du serveur.
- 

## **GET /collections/collection/ :collection\_id/review**

### **Description**

Récupère les flashcards à réviser dans une collection pour l'utilisateur connecté, selon le système de répétition espacée.

### **Authentication requisite**

Oui — JWT valide. L'utilisateur doit être le propriétaire de la collection.

### **Middleware**

- `authenticateToken` : vérifie la présence et la validité du JWT.
- `validateParams` : valide les paramètres de la route.

### **Validation (Zod)**

- `collection_id` : UUID (paramètre de route).

### **Headers**

- `Authorization`: Bearer <token>

### **Paramètres**

- `collection_id` (route param, UUID) : identifiant de la collection.



#### Body attendu

Aucun.

#### Réponse – Succès (200)

```
[
 {
 "id": "f1c1b1a1-9f9d-4c8c-9c1c-8d8d1f1c9a0a",
 "front_text": "Recto de la carte à réviser",
 "back_text": "Verso de la carte à réviser",
 "collection_id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "progress_level": 2,
 "last_review": "2024-01-08T10:00:00.000Z",
 "next_review_date": "2024-01-10T10:00:00.000Z"
 }
]
```

#### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide.
  - 403 Forbidden : l'utilisateur n'est pas autorisé.
  - 404 Not Found : collection non trouvée ou aucune flashcard à réviser.
  - 500 Internal Server Error : erreur interne du serveur.
- 

### 3 Flashcards

Les endpoints Flashcards permettent de créer, consulter, modifier, supprimer et réviser les flashcards. Toutes les routes nécessitent **un utilisateur authentifié** via JWT.

---

#### GET /flashcards/ :id

##### Description

Récupère une flashcard par son identifiant.

##### Authentication requise

Oui — JWT valide.

##### Middleware

- authenticateToken : vérifie la présence et la validité du JWT
- validateParams : valide les paramètres de la route.

##### Validation (Zod)

- id : UUID valide (paramètre de route)

##### Headers

- Authorization: Bearer <token>

##### Paramètres

- id (route param, UUID) : identifiant de la flashcard

#### Body attendu

Aucun.

#### Réponse – Succès (200)

```
{
 "id": "536cf903-7bd7-4279-9c2b-0c013b4d68f5",
 "front_text": "Paris",
 "back_text": "France",
 "url_front": "https://www.okvoyage.com/wp-content/uploads/2023/10/Paris-en-photos-scaled.jpg",
 "url_back": "https://c8.alamy.com/compfr/g2xyg1/carte-vectorielle-detaillée-de-la-france-et-capitale-par"
```

```

 "collection_id": "90d1a062-7570-4f8a-b200-1f67357e3d3d",
 "createdAt": "2026-01-10T10:27:47.000Z"
 }

```

### Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod)
- 401 Unauthorized : token manquant, expiré ou invalide
- 403 Forbidden : utilisateur non autorisé
- 404 Not Found : flashcard non trouvée
- 500 Internal Server Error : erreur interne du serveur

## POST /flashcards/

### Description

Crée une nouvelle flashcard dans une collection.

### Authentication requise

Oui — JWT valide.

### Middleware

- authenticateToken : vérifie la présence et la validité du JWT
- validateBody : valide le corps de la requête.

### Validation (Zod)

- front\_text : string non vide de taille entre 1 et 512 caractères
- back\_text : string non vide de taille entre 1 et 512 caractères
- url\_front : url valide optionnel
- url\_back : url valide optionnel
- collection\_id : UUID valide - identifiant de la flashcard

### Headers

- Authorization: Bearer <token>

### Paramètres

Aucun.

### Body attendu

```

{
 "front_text" : "Fabienne",
 "back_text" : "Jort",
 "url_front" : "https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Ffr.wikipedia.org%2Fwik
 "collection_id" : "1dcde502-99b3-4294-809d-5fa854218890"
}

```

### Réponse – Succès (200)

```

{
 "message": "Flashcard created",
 "data": {
 "id": "39bbcb6-7ce3-4822-89d9-c5bcd34e5a7",
 "front_text": "Fabienne",
 "back_text": "Jort",
 "url_front": "https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Ffr.wikipedia.org%2Fwik
 "url_back": null,
 "collection_id": "1dcde502-99b3-4294-809d-5fa854218890",
 "createdAt": "2026-01-10T10:59:40.000Z"
 }
}

```

### Erreurs possibles

- 400 **Bad Request** : données invalides (échec de la validation Zod)
  - 401 **Unauthorized** : token manquant, expiré ou invalide
  - 403 **Forbidden** : utilisateur non autorisé
  - 404 **Not Found** : flashcard non trouvée
  - 500 **Internal Server Error** : erreur interne du serveur
- 

## DELETE /flashcards/ :id

### Description

Supprime une flashcard existante.

### Authentication requise

Oui — JWT valide, propriétaire ou admin.

### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT
- `validateParams` : valide les paramètres de la route.

### Validation (Zod)

- `id` : UUID valide (paramètre de route)

### Headers

- `Authorization`: Bearer <token>

### Paramètres

- `id` UUID valide : identifiant de la flashcard

### Body attendu

Aucun.

### Réponse – Succès (200)

```
{
 "message": "Flashcard deleted !"
}
```

### Erreurs possibles

- 400 **Bad Request** : données invalides (échec de la validation Zod)
  - 401 **Unauthorized** : token manquant, expiré ou invalide
  - 403 **Forbidden** : utilisateur non autorisé
  - 404 **Not Found** : flashcard non trouvée
  - 500 **Internal Server Error** : erreur interne du serveur
- 

## PATCH /flashcards/ :id

### Description

Modifie une flashcard existante (front, back, URLs).

### Authentication requise

Oui — JWT valide, propriétaire ou admin.

### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT
- `validateParams` : valide les paramètres de la route.
- `validateBody` : valide le corps de la requête.

### Validation (Zod)

- `id` : UUID valide (paramètre de route)
- `front_text` : string non vide de taille entre 1 et 512 caractères optionnel

- `back_text` : string non vide de taille entre 1 et 512 caractères optionnel
- `url_front` : url valide optionnel
- `url_back` : url valide optionnel
- Il faut au moins un des quatre paramètres soit modifié

#### Headers

- `Authorization: Bearer <token>`

#### Paramètres

- id UUID valide : identifiant de la flashcard

#### Body attendu

```
{
 "url_front" : "https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Ffr.wikipedia.org%2Fwik
```

#### Réponse – Succès (200)

```
{
 "message": "Flashcard updated !"
}
```

#### Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod)
- 401 Unauthorized : token manquant, expiré ou invalide
- 403 Forbidden : utilisateur non autorisé
- 404 Not Found : flashcard non trouvée
- 500 Internal Server Error : erreur interne du serveur

### PATCH /flashcards/revise/ :id

#### Description

Enregistre une révision d'une flashcard et met à jour son niveau et la date de prochaine révision.

#### Authentification requise

Oui — JWT valide.

#### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT
- `validateParams` : valide les paramètres de la route.
- `validateBody` : valide le corps de la requête.

#### Validation (Zod)

- `progress_level` : nombre entier entre 1 et 5

#### Headers

- `Authorization: Bearer <token>`

#### Paramètres

- id UUID valide : identifiant de la flashcard

#### Body attendu

```
{
 "progress_level" : 3
}
```

#### Réponse – Succès (200)

```
{
 "message": "Progression updated",
 "data": {
```

```

 "flashcard_id": "54d1ff1a-decb-4ede-a862-e7f60355972e",
 "user_id": "1d14e53a-354f-4077-b0c9-1af6c5ba24fa",
 "progress_level": 3,
 "last_review": "2026-01-10T11:10:30.000Z",
 "next_review_date": "2026-01-14T11:10:30.000Z"
 }
}

```

#### Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod)
- 401 Unauthorized : token manquant, expiré ou invalide
- 403 Forbidden : utilisateur non autorisé
- 404 Not Found : flashcard non trouvée
- 500 Internal Server Error : erreur interne du serveur

### GET /flashcards/reviewAll

#### Description

Récupère toutes les flashcards à réviser pour l'utilisateur connecté, toutes collections confondues.

#### Authentification requise

Oui — JWT valide.

#### Middleware

- authenticateToken : vérifie la présence et la validité du JWT

#### Validation (Zod)

Aucune.

#### Headers

- Authorization: Bearer <token>

#### Paramètres

Aucun.

#### Body attendu

Aucun.

#### Réponse – Succès (200)

```

[
 {
 "id": "f1c1b1a1-9f9d-4c8c-9c1c-8d8d1f1c9a0a",
 "front_text": "Recto de la carte à réviser",
 "back_text": "Verso de la carte à réviser",
 "collection_id": "c1f7a2d1-8e4d-4b8b-8e1e-7d7b0f1c9a0a",
 "progress_level": 2,
 "last_review": "2024-01-08T10:00:00.000Z",
 "next_review_date": "2024-01-10T10:00:00.000Z"
 }
]

```

#### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide.
- 404 Not Found : Aucune flashcard à réviser.
- 500 Internal Server Error : erreur interne du serveur.

## 4 Utilisateurs (Admin)

Ces endpoints permettent à un administrateur de gérer les utilisateurs.  
Toutes les routes nécessitent **un JWT valide** et **un rôle admin**.

---

### GET /users

#### Description

Liste tous les utilisateurs triés par date de création (les plus récents en premier).

#### Authentification requise

Oui — JWT valide et rôle **admin**.

#### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT
- `isAdmin` : vérifie que l'utilisateur est admin

#### Validation (Zod)

Aucun.

#### Headers

- `Authorization`: Bearer <token>

#### Paramètres

Aucun.

#### Body attendu

Aucun.

#### Réponse – Succès (200)

```
[
 {
 "id": "1d14e53a-354f-4077-b0c9-1af6c5ba24fa",
 "email": "test@test.com",
 "firstname": "alexandre",
 "lastname": "LeRoy",
 "admin": false,
 "createdAt": "2026-01-10T10:27:47.000Z"
 },
 {
 "id": "44582624-118a-4c3c-8f58-a170c2e00308",
 "email": "test2@test.com",
 "firstname": "Cyprien",
 "lastname": "Duroy",
 "admin": true,
 "createdAt": "2026-01-10T10:27:47.000Z"
 },
 {
 "id": "b7b64bef-6eff-4d6a-b32f-3e412d7cffbd",
 "email": "test3@test.com",
 "firstname": "Louis",
 "lastname": "Martin",
 "admin": false,
 "createdAt": "2026-01-10T10:27:47.000Z"
 }
]
```

#### Erreurs possibles

- 401 Unauthorized : token manquant, expiré ou invalide

- 403 Forbidden : utilisateur non autorisé
  - 404 Not Found : utilisateur non trouvée
  - 500 Internal Server Error : erreur interne du serveur
- 

## GET /users/ :id

### Description

Récupère les informations d'un utilisateur spécifique.

### Authentication requise

Oui — JWT valide et rôle `admin`.

### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT
- `isAdmin` : vérifie que l'utilisateur est admin
- `validateParams` : valide les paramètres de la route.

### Validation (Zod)

- `id` : UUID valide (paramètre de route)

### Headers

- `Authorization: Bearer <token>`

### Paramètres

- `id` UUID valide : identifiant de la personne

### Body attendu

Aucun.

### Réponse – Succès (200)

```
{
 "id": "1d14e53a-354f-4077-b0c9-1af6c5ba24fa",
 "email": "test@test.com",
 "firstname": "alexandre",
 "lastname": "LeRoy",
 "admin": false,
 "createdAt": "2026-01-10T10:27:47.000Z"
}
```

### Erreurs possibles

- 400 Bad Request : données invalides (échec de la validation Zod)
  - 401 Unauthorized : token manquant, expiré ou invalide
  - 403 Forbidden : utilisateur non autorisé
  - 404 Not Found : utilisateur non trouvée
  - 500 Internal Server Error : erreur interne du serveur
- 

## DELETE /users/ :id

### Description

Supprime un utilisateur et gère les conséquences sur ses collections et flashcards.

### Authentication requise

Oui — JWT valide et rôle `admin`.

### Middleware

- `authenticateToken` : vérifie la présence et la validité du JWT
- `isAdmin` : vérifie que l'utilisateur est admin
- `validateParams` : valide les paramètres de la route.

**Validation (Zod)**

- id : UUID valide (paramètre de route)

**Headers**

- Authorization: Bearer <token>

**Paramètres**

- id UUID valide : identifiant de la personne

**Body attendu**

Aucun.

**Réponse – Succès (200)**

```
{
 "message": "Utilisateur et toutes ses données associées supprimés avec succès.",
 "userId": "1d14e53a-354f-4077-b0c9-1af6c5ba24fa"
}
```

**Erreurs possibles**

- 400 Bad Request : données invalides (échec de la validation Zod)
- 401 Unauthorized : token manquant, expiré ou invalide
- 403 Forbidden : utilisateur non autorisé
- 404 Not Found : utilisateur non trouvée
- 500 Internal Server Error : erreur interne du serveur

---

Document réalisé par Duroy Cyprien et Le Roy Alexandre